

Snowflake - Professional Training

Facilitator: Ramkumar J Dhamodharan

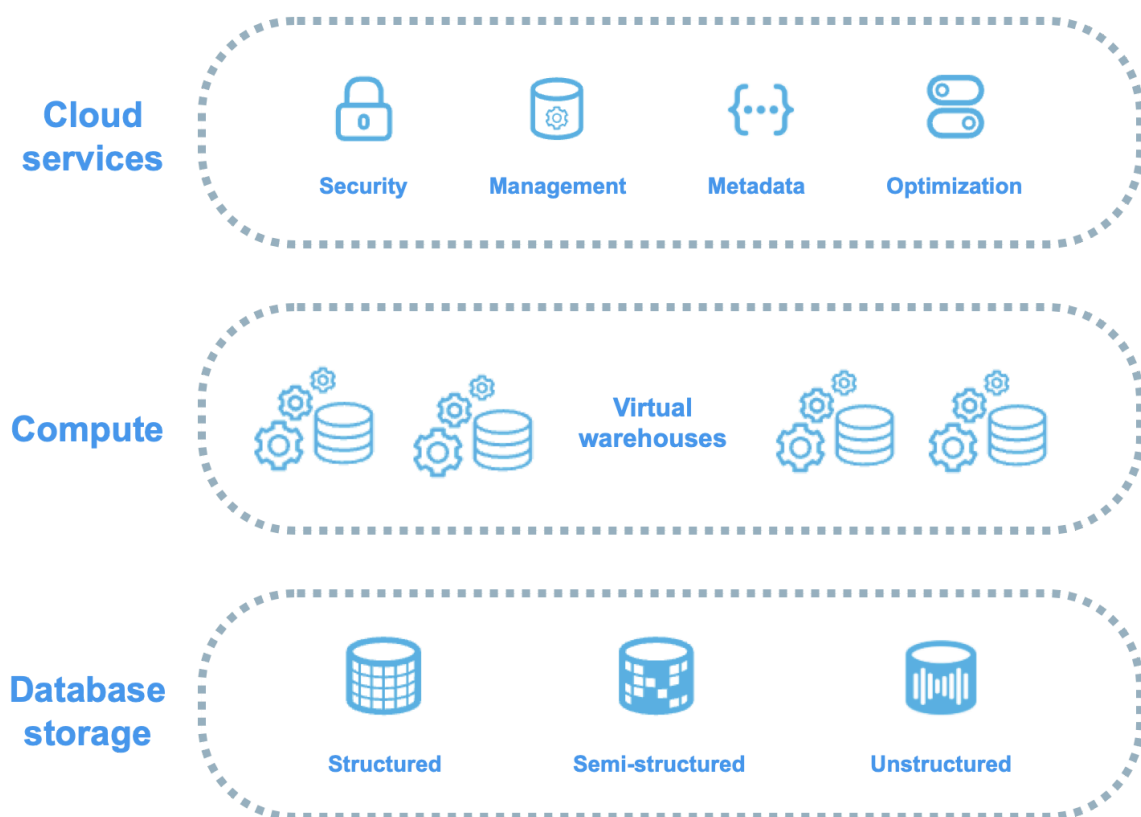
<https://www.linkedin.com/in/ramkumar-j-d-57423611/>

Day Agenda

- Lecture on Snowflake Essentials
- Architecture
- Virtual Warehouse - Cost & Performance
- Database, Schema and Modeling
- Data Ingestion Techniques
 - Batch
 - Continuous
 - Streaming
 - Semi-Structured
- Demonstration & Practices

Snowflake Architecture

- In traditional warehouses, tie Storage and Compute together.
- Snowflake physically separates these into three different parts
 - Storage
 - Compute
 - Cloud Services
- Storage - Cloud Object Storage - Compressed - Immutable Files
- Compute - Virtual Warehouses - which are just clusters of computers you spin up and down.
- Cloud Services - The brain - Optimizer, Metadata, Security and Transaction Management



Analogies:

- Storage - Library
- Warehouses - Reading Rooms
- Cloud Services - Librarian

Micro-Partitions

- Small, Immutable Chunks of roughly 50 - 500 MB of uncompressed data each
- It remembers the min, max value of every column in every chunk
- WHERE trade_ts >= '2024-06-04'
- The phrase to remember here is that "PARTITIONS SCANNED vs. PARTITIONS TOTAL"

Warehouses

- The Cost & Performance Dial
- T-Shirt Size - XS, S, M, L, XL & Up
- Reason through
 - Credits Per Hour
 - Credits Per Query
- What size? And how long does it stay awake?
- Two Independent Knobs
 - Scale Up (More Power for a single heavy query)
 - Scale Out (Multi-Cluster - More copies of the same warehouse that spin up under concurrency)
- Analogy
 - Match the level to the symptom
 - One Slow Query -> Bigger Truck
 - A queue of waiting queries - More Trucks
- Two Settings
 - Auto-Suspend
 - Auto-Resume

The Lesson - Aggressive suspension is for SPIKY workloads with gaps - For a steady trickle, let it stay warm.

Database, Schemas and Modeling for Engineering

- Hierarchy
 - Account -> Database -> Schema -> Objects
 - (Tables, Views, Stages, Pipes, File Formats, Integrations ...)
 - FQN (Fully Qualified Name - Database.Schema.Table)
- Layered Architecture
 - Medallion type
 - Raw | Staging | Analytics
 - Raw - Data lands here exactly as it arrives - source of truth
 - Staging - Cleaned | Typed | Deduplicated | Conformed (We trust this now - Layer)

- Analytics - Curated, Business-Ready Tables & Views that BI team and analysts actually query

Table Types

- Permanent - The Default - Has time-travel, Fail-Safe - Use for anything that matters
- Transient - No Fail-Safe, Shorter Time-Travel => Cheaper Storage
- Temporary - Vanishes when your session ends.

Can you reload the landing tables from S3 buckets? Yes, make them transient.

Snowflake's columnar storage just makes the wider dimensions join cheap.

OLTP systems - Row Storage

OLAP systems - Column Storage

Getting Data In

- The Bulk Loading Mental Model

Door	Tool	When	Data Unit
Batch Files / On-Demand	COPY INTO	Load this pile of files now	File(s)
Files, Continuously	Snowpipe	Load each file as it lands	File(s)
Rows, Continuously	Kafka Snowpipe Streaming	Load events in near real-time	Row(s)

For bulk, the key concept is the stage - A pointer to where files live.

Storage Integration - Secure Handshake - Is a trust relationship between your snowflake account and an AWS IAM role.

Step by Step Process

- Create a storage integration in snowflake pointing at your bucket
- Run describe integration - which shall give you IAM User ARN and External ID
- In AWS, configure IAM role, trust policy and provide those two values
- Make sure that the role itself has S3 read permissions on the bucket
- In snowflake, create the stage that uses the integration

The three classic load failures

- When a load fails, it's almost always one of the three things
 - The file format does NOT match the file
 - A column count mismatch
 - A date/number that won't parse
- Load History (It remembers the metadata of what files have been loaded earlier)
- Load History Window

Semi-Structured Data - JSON without the pain

- VARIANT
- A single column that holds the entire JSON document (or array or any semi-structured value)
- `SELECT raw:symbol::string, raw:price::float from market_ticks_json;`
- `LATERAL_FLATTEN` - helps to explode an array of values into rows - turning one nested document into a proper table

- Load Now, define the shape later - This is SCHEMA-ON-READ

Snowpipe - Continuous File Ingestion

- Snowpipe is a COPY that fires automatically, the moment a file appears.
- Same load logic, but the trigger is the file's arrival, not your hand on the keyboard
- How does Snowpipe really work?
 - A file lands in the S3 bucket
 - S3 fires an event notification to an SQS queue (Snowflake gives you the Queue ARN to point your buckets' event notification at)
 - Snowpipe of Snowflake sees the message and runs the COPY command for that one file, using the pipe's predefined COPY command
 - Rows show up in your table seconds-to-minutes later - SERVERLESS
 - No Warehouse your manage | Pay to keep warm
- Snowpipe has a per-file cost component, so the shape of your files matters a lot.

Kafka & SnowPipe Streaming - True Real-Time

- How does it work?
 - Kafka topic carries the market-tick events
 - The Snowflake Sink Connector running in your Kafka cluster subscribes to the topic and writes straight into the Snowflake table
 - An option (ingestion method = SNOWFLAKE STEAMING vs. SNOWPIPE) - This configuration is the difference between rows-direct and the old-batch-into-files behavior
 - Authentication is KEY-PAIR (private and public key) - the connector signs in with the private key, and Snowflake verifies it against the public key you registered on the user.

Atlas - Fidelity Investments

- Fidelity's E&O (Equity and Options) Trading Desk is consolidating its market data estate into Snowflake.
- The codename for this platform is ATLAS
 - Must ingest market activity at three different velocities and serve it to analysts, risk and compliance from one place
 - Historical (Years of Equity Ticks for back-testing and TCA)
 - Batch (Nightly trade files dropped by venues (NYSE) into S3)
 - Micro-Batch (Intraday tick files that land through the day and must be loaded within seconds)
 - Streaming (A live execution feed (orders/fills) flowing through Kafka, landing in Snowflake with low latency (Snowpipe Streaming))
- Along the way, the data is MESSY semi-structured JSON (nested option chains) that must be modeled into clean STAR schemas

Mermaid Diagram Contents

Browser -> <https://mermaid.live>

```
graph TD
    subgraph Sources ["Market Data Sources"]
        NYSE["NYSE / NASDAQ<br/>nightly batch CSV"]
        CBOE["CBOE options<br/>nested JSON"]
        VENUE["intraday tick drops"]
        EXEC["live execution feed"]
    end

    subgraph AWS ["AWS Landing Zone"]
        S3BATCH["S3 /batch (Lab 4)"]
        S3JSON["S3 /json (Lab 5)"]
        S3PIPE["S3 /autoingest (Lab 6)"]
        SQS["SQS queue"]
    end

    subgraph DOCKER ["Local Docker (Lab 7)"]
        KAFKA["Kafka broker + Connect"]
    end

    subgraph SF ["Snowflake DB: FIDELITY_ATLAS"]
        subgraph COMPUTE ["Compute - Labs 1 & 2"]
            WH["Multi-cluster warehouses<br/>pruning-aware queries"]
        end

        subgraph RAW ["RAW layer"]
            RTICKS["TICKS clustered (Lab 1)"]
            RTRD["RAW_TRADES (Lab 4)"]
            RJJSON["RAW_OPTIONS VARIANT (Lab 5)"]
            RDROP["RAW_TICKS_STREAM (Lab 6)"]
            RSTREAM["RAW_EXECUTIONS (Lab 7)"]
        end

        subgraph MODEL ["ANALYTICS layer (Lab 3)"]
            DIM["DIM_SECURITY / DIM_VENUE / DIM_DATE"]
            FACT["FACT_TRADES / FACT_QUOTES / FACT_EXECUTIONS"]
        end
    end

    subgraph CONS ["Consumers"]
        AN["Trading analysts"]
        RK["Risk & compliance"]
    end
```

end

NYSE --> S3BATCH --> RTRD
CBOE --> S3JSON --> RJSON
VENUE --> S3PIPE --> SQS --> RDROP
EXEC --> KAFKA --> RSTREAM

RTICKS --> MODEL
RTRD --> MODEL
RJSON --> MODEL
RDROP --> MODEL
RSTREAM --> MODEL
WH -. -> RAW
WH -. -> MODEL
MODEL --> AN
MODEL --> RK

The following are the capabilities we would like to experiment with.

- Pruning Experiments (Why are tick queries slow?)
- Scaling & Concurrency Test (Market open overwhelms the desk)
- The Star-Schema Modeling Challenge (Analysts can't read raw feeds)
- Bulk Load + S3 Integration
- Semi-Structured JSON
- SnowPipe Auto-Ingest (SQS) - (Nightly is too slow)
- Kafka | Snowpipe Streaming (The execution feed must stream - real-time?)

Day 2 Agenda

- Demonstrations of Day 1 Contd.,
- Lecture on
 - Streams (CDC)
 - Tasks | Scheduling
 - Dynamic Tables
 - Apache Airflow (Enterprise Orchestration - Snowflake Integration)
 - Advanced SQL
 - Query Performance Tuning
 - Clustering, Search Optimization, Materialized Views
 - Caching & Cost Governance
- Demonstrations & Practices

Streams (CDC - Change Data Capture)

How do we process ONLY what's New?

- The Concept: Stream
- A stream is a bookmark on a table
- It does NOT copy the data - It remembers an offset (a point in the tables' change history) and shows you everything that changed since you last looked
- INSERTS, UPDATES, DELETES each tagged with metadata columns
- A stream is the UNREAD-MESSAGES badge on your inbox. Reading the messages (consuming the stream in a DML statement) clears the badge - Just peeking with a SELECT does NOT.
- Streams can go stale if UNCONSUMED past the tables' retention window.

Tasks - Native Scheduling

We need scheduled execution that lives WHERE the data lives, with history we can audit.

- The concept: Task
- A task is a scheduled SQL statement (or procedure call) inside Snowflake.
- Tasks chain into DAGs - a root task on a schedule, children firing AFTER parents.
- Two flavors
 - Warehouse Tasks (You choose the warehouse)
 - Serverless (Snowflake sizes compute per run)
- DAG - Directed Acyclic Graphs
 - WHEN SYSTEM\$STREAM_HAS_DATA('stream-name')
- TASK_HISTORY
- Scheduled
- Conditional on real-changes
- Fully Audited
- No VM

Dynamic Tables - The declarative alternative

Can we just declare WHAT each table should contain and let Snowflake figure out HOW to keep it fresh?

- Managing Streams + Tasks + MERGE + Dependency Graph for complex scenarios require a good amount of review / maintain / operations

How do we simplify this?

- Concept: Dynamic Tables
- A dynamic table is a SELECT statement plus a freshness promise.

CREATE DYNAMIC TABLE positions

TARGET_LAG = '5 minutes'

WAREHOUSE = 'DEMO_WH'

AS

SELECT ...;

- Snowflake watches the sources, computes incremental refreshes itself, and chains dynamic tables BUILT on dynamic tables into an automatic dependency graph
- STREAMS + TASKS = Manual Gear Car
- DYNAMIC TABLES = Automatic Gear Car

Real-World Scenario:

For a scenario of a 40-table analytics layer, dynamic tables collapse the plumbing into ONE declaration per table.

Apache Airflow - Enterprise Orchestration Engine

The compliance team wants ONE place to see every overnight job across the firm - NOT ONE CONSOLE PER TOOL.

- The concept: Airflow
- Airflow is an OPEN-SOURCE, LICENSE-FREE (Apache License) - Light Weight - a general purpose orchestrator - Pipelines are Python DAGs - Steps are OPERATORS (Run SQL, Call an API, Run Python, Wait on a sensor ...) with retries, alerting, backfills, and ONE UI for everything
- Rule of Thumb
 - In-Database, Data Driven Steps -> Native Tasks | Dynamic Tables
 - Cross-system workflows to notify humans -> Airflow

Advanced SQL Capabilities

- Window Functions
- MERGE
- Procedures

Three Important Tools, one each

- Window Functions (Compute across rows without collapsing them - Aggregate functions OVER clause using partitions and ordering)
 - QUALIFY for Deduplication
- MERGE
 - The update or insert in one atomic statement
- Stored Procedures
 - Snowflake Scripting
 - Wire the multi-step logic with variables, control flow and exception handling into ONE callable unit (CALL xxx)

Query Performance Tuning - Strategies

- Fidelity needs engineers who diagnose before they spend.
- The diagnostic instrument is the QUERY PROFILE
- Three patterns look after in the QUERY PROFILE window
 - Poor Pruning
 - Spillage
 - Join Explosion
- Poor Pruning - Partitions scanned vs. Partitions Total
 - The filter column is NOT organized
 - Fix: Clustering
 - NOT A BIGGER WAREHOUSE
- Spillage
 - Bytes spilled to Local / Remote Storage
 - Fix: BIGGER WAREHOUSE
 - Or restructure the query
- Join Explosion
 - A join node EMITS far more rows than it receives
 - DUPLICATE JOIN KEYS MULTIPLYING ROWS
 - Fix: The Join Logic
 - NO HARDWARE HELPS

Clustering, Search Optimization and Materialized Views

- One table, three problems - There's NO single fix.

The decision triangle

- Clustering key
 - Range Scans
 - Reorganizes micro-partitions around your key
 - CLUSTER_KEY
 - Background reclustering costs credits - worth it for the DOMINANT filter.
- Search Optimization
 - Point Lookups
 - A persistent search structure for EQUALITY predicates
 - Find **ACCT83498** goes from FULL-SCAN to INDEX-LIKE SEEK
 - Pay STORAGE + MAINTENANCE, But it's POINTLESS for RANGE SCANS.
- Materialized Views
 - Repeated Aggregations
 - Pre-compute the rollup once, hundreds of dashboard hits read the stored result, AUTO KEPT-REFRESH.
 - Pay maintenance on EVERY BASE TABLE CHANGE
 - WRONG for rarely read aggregations OVER HOT-TABLES (B's Rows)

Caching and Cost Governance

- Fidelity needs attribution (who spends what? / How much? when) - and circuit breakers (hard limits)
- Three Layers
 - Caches
 - The result cache returns repeated identical queries in milliseconds at ZERO COMPUTE
 - Warehouse local cache makes warm warehouses faster
 - CURRENT_TIMESTAMP in a query defeats the whole purpose of RESULT CACHE
 - Resource Monitors
 - The circuit breaker
 - Credit quota per warehouse / account with thresholds
 - Attribution
 - ACCOUNT_USAGE
 - Query Tagging
 - QUERY_HISTORY
 - It helps to rank expensive queries
 - WAREHOUSE_METERING_HISTORY
 - Splits spends by warehouse

Day 3 Agenda

- Demonstration & Practices - Day 2 Topics
 - Caching and Cost Governance
- Lecture on Advanced Topics
 - Time Travel & Zero Copy Cloning
 - Secure Data Sharing
 - RBAC (Role Based Access Control)
 - Data Security (Masking, RLS (Row Level Security) & Tags)
 - Security & Audit
 - Snowpark
 - Migration & Engineering Disciplines
 - Agentic AI on Snowflake
 - ML
 - Generative AI
 - Agentic AI
- Demonstration & Practices

Day 3 Lecture Topics

Time Travel & Zero Copy Cloning

- Time Travel
 - Every change to a table is retained for a window (1 day default, up to 90 days for enterprise edition)
 - Query the past directly using AT operator | BEFORE
 - AT(OFFSET >= -60*5)
 - BEFORE(statement <> 'query_id')
 - UNDROP
 - Resurrects the whole object
 - The fix for the WHERE-less update is one statement - RE-INSERT from the table as it was before that query id in seconds, NOT hours.
- Zero Copy Cloning
 - CREATE TABLE <X> CLONE <Y>
 - Duplicates metadata pointers, NOT data - INSTANT, NEAR-ZERO storage
 - The clone only pays for blocks that diverge after cloning
 - Full production shaped dev/test environments stop being a storage negotiation

Secure Data Sharing

- There has to be a way to grant LIVE, read-only, revocable access without the data ever moving
- The concept: A Share
- A share grants another Snowflake account read access to your storage - NO COPY, NO MOVEMENT.
- The consumer queries live data with their compute environment, and you can revoke the access instantly.
- Wrap exposure in secure views so they see exactly the columns/rows you choose and nothing about the underlying definition.
- For partners without Snowflake reader accounts, you provision a managed account they log into, or compute you control.

RBAC (Role Based Access Control)

- The model: privileges attach to ROLES, roles to USERS, roles to ROLES (Hierarchy)
- This discipline that scales -
 - Two layers of custom roles
 - Access Roles
 - What
 - Bundling of privileges on objects
 - Functional Roles
 - Who
 - Personas (Advisor, Analyst, Compliance), granted access roles, granted to humans
- User -> Advisor -> Database_object_READ -> SELECT on [OBJECT]

- Two pointers to remember
 - Ownership (whoever creates an object owns it - create with the right role or transfer)
 - Future Grants (without them, every new table silently lacks the grants and tickets pile up)

Masking, Row Access & Tags

- RBAC says who can touch the table - but an enterprise needs control INSIDE the table - Which columns, which rows automatically.
- Three instruments
 - Dynamic Data Masking
 - A policy on the column
 - Same query - different results by role
 - Data never changes, but the view of it does
 - For example, compliance team sees the column value as 12345678 but others see *****
 - Row Access Policies
 - A predicate on the rows
 - The filter lives in the table, not in every query someone remembers to write.
 - Tag based Masking
 - Classify once
 - For example, PII_TYPE=SSN tag
 - Attach the masking policy to the tag, any future column tagged PII is MASKED automatically the moment they're tagged - You don't have to explicitly manage the masking

Security & Audit

- The enterprise needs forensic-grade access history and credential hygiene
- 3 Important features
 - Access History
 - ACCESS_HISTORY - information management view, column level forensics - who read which columns of which tables, when via which query
 - Key pair Authentication
 - Service accounts authenticate with a private key - No Passwords - No Leak
 - Network Policies
 - IP Allow Lists - Per User / Account
- SSO | MFA
- Encryption Everywhere

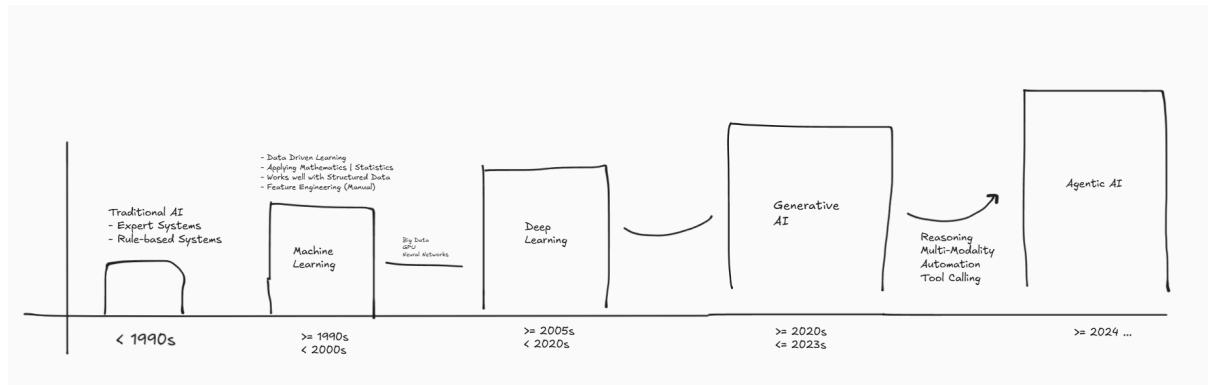
Snowpark

- Snowpark is a developer framework by Snowflake that allows you to build data pipelines and machine learning (ML) workflows using languages like Python, Java, and Scala directly within Snowflake's engine. It eliminates the need to extract, move, or transfer data to external servers, ensuring processing speed and robust data governance.
- Send the code to the data where it lives
- Snowpark inverts it
- (Usually, the data is brought from the database to the code where the compute happens)
- The Python's Data Frame API looks like Pandas but lazily compiles to SQL running on your Snowflake warehouse, so nothing leaves Snowflake
- The volatility job becomes a Python procedure running where the data lives - No Export, No VM, Full Governance

Migration and Engineering Principles | Disciplines

- Prefer to use Migration services provided out of the box in Snowflake which supports well-known database migration services (SQL Server, Postgres and Oracle)
- When it comes to Legacy Warehouse | Scripts, The simple "Lift the SQL over" won't work out
- Convert a deliberately quirky generic-legacy script to Snowflake SQL - Importantly apply reconciliation process to Row Counts, Sums, Checksums between old-logic and new-logic outputs
- Plus, the habits such as
 - SQL in Git Version Control
 - Environments promoted Dev-> Test -> Prod (Zero-Copy clones make test environments free)
 - Schema changes as reviewed migrations
 - The Validation Queries live in CI/CD pipelines
 - DQ-Gate Patterns when they're orchestrated using Airflow | other tools

AI Evolution



Snowflake AI Stack

- Cortex AISQL Functions
 - Pre-built LLMs as functions
 - CLASSIFY
 - SENTIMENT
 - COMPLETE
- **Semantic Views + Cortex Analyst**
 - Grounded
 - Not Guessing based
- Cortex Search
 - Documents + RAG
- Cortex Agents
 - The orchestration platform combining Semantic Views + Cortex Analyst + Cortex Search & Tool calling abilities
- Cortex Code (CoCo) - SQL Code Explanation | Fixing | Generation - End-user agent surfaces for business users and developers perform SQL related interactive operations.